



**School of Future Tech**

## **Case Study Report**

on

**GAME ENGINE USING C++**

by

**ANTARA YOGESH PALWANKAR**

**150096725126**

**SAM ALTMAN COHORT**

# **INDEX:**

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study
5. Problem Statement / Case Study Implementation Details and Snapshots
6. Problem Statement / Case Study Results and Conclusion
7. References

## **1. INTRODUCTION TO THE CASE STUDY:**

Game development combines creativity with programming logic. A game engine is a software framework that manages important game components such as rendering, object management, input handling, and game logic. Instead of building everything from scratch for each game, developers use a game engine to control how the game operates.

This case study focuses on building a basic game engine framework using C++. The project demonstrates how core programming concepts such as classes, loops, random behaviour, and collision detection can be used to build a simple interactive console-based game. The player moves within a grid environment while enemies move randomly, and the engine manages the interaction between them.

## **2. PROBLEM STATEMENT / CASE BACKGROUND:**

### **BACKGROUND:**

Modern video games rely on powerful game engines to manage different systems like graphics, physics, and input processing. Understanding these systems can begin with building a simplified engine that focuses on the basic mechanics of a game.

A simple grid-based game allows developers to understand how game loops work, how objects interact with each other, and how user input affects the game environment.

### **ABSTRACT:**

This case study presents the design and implementation of a basic game engine using C++. The system includes a game loop, object management system, input handling, and collision detection. A player object moves within a grid while enemy objects move randomly.

Whenever the player collides with an enemy, the enemy is defeated and the score increases. The project demonstrates how object-oriented programming and simple algorithms can be used to create a functional game framework that manages multiple game elements.

### **3. PROBLEM STATEMENT / CASE STUDY DESIGN:**

The system is designed using a modular structure, where different components manage specific tasks within the game.

#### **Game Loop:**

The game runs using a continuous loop that repeatedly performs three main operations:

- Rendering the game grid
- Processing player input
- Updating the game state

#### **Object Management:**

All game elements are represented using a GameObject class, which stores position and display symbols. The main objects include:

- Player object
- Enemy objects

#### **Input Handling:**

Player movement is controlled using keyboard keys:

- W – Move up
- S – Move down
- A – Move left
- D – Move right
- Q – Quit the game

#### **Collision Detection:**

If the player moves into the same grid position as an enemy, a collision occurs and the enemy is defeated.

#### **Score System:**

Each defeated enemy increases the player's score.

#### **4. METHODS AND ALGORITHMS TECHNOLOGY APPLIED IN THE PROBLEM STATEMENT / CASE STUDY:**

##### **Object-Oriented Programming:**

The game is implemented using object-oriented programming principles. The GameObject class represents entities such as the player and enemies, allowing reusable and organised code.

##### **Random Movement Algorithm:**

Enemy movement is controlled using a random direction generator. Each enemy randomly moves in one of four directions:

- Up
- Down
- Left
- Right

The direction is generated using the rand() function, making enemy movement unpredictable.

##### **Collision Detection:**

Collision detection checks whether the player and an enemy occupy the same position on the grid. When both coordinates match, the enemy is defeated and the score increases.

##### **Technology Stack Used:**

Programming Language

- C++

##### **Libraries Used:**

- iostream – input/output operations
- vector – storing enemy objects
- cstdlib – random number generation
- c time – random seed initialisation

##### **Development Environment:**

- Visual Studio Code or any C++ IDE
- Console-based execution

## 5. PROBLEM STATEMENT / CASE STUDY IMPLEMENTATION DETAILS AND SNAPSHOTS:

The implementation of the game engine is divided into different components to manage the game efficiently.

### GameObject Class:

This class represents objects in the game environment. Each object stores its x and y position along with a symbol used for display. Both the player and enemies are created using this class.

### Game Class:

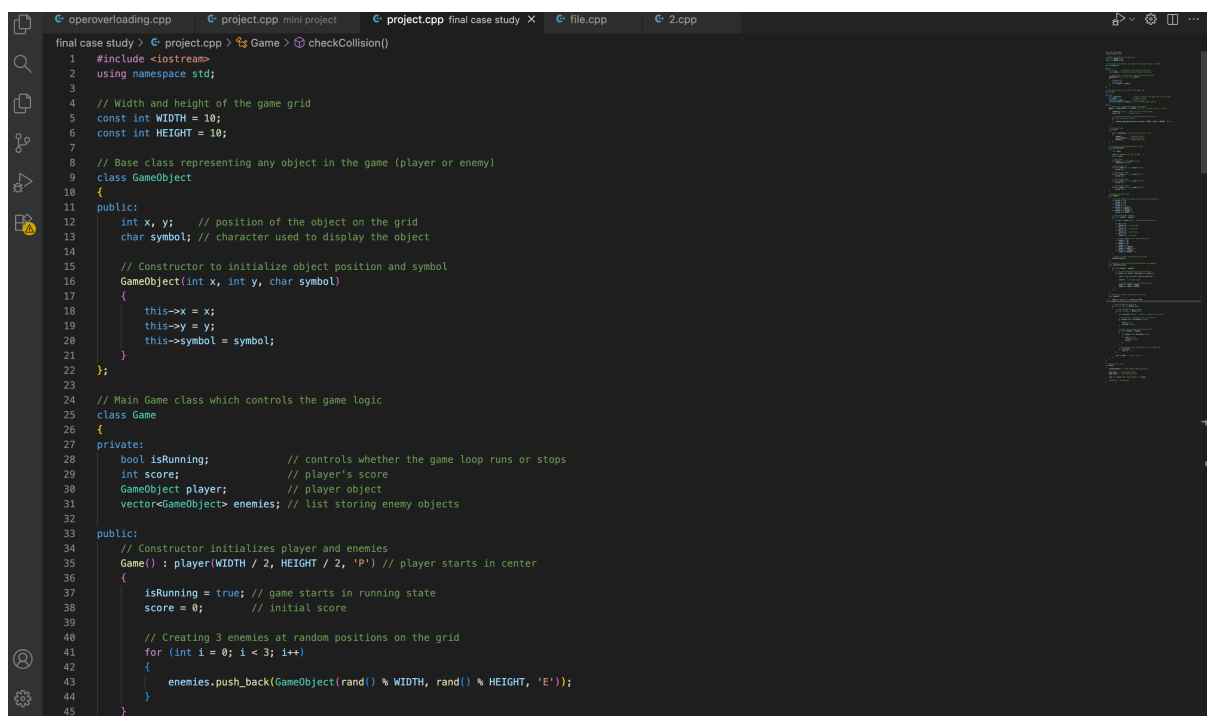
The Game class controls the entire game system. It manages the game loop, input handling, enemy movement, collision detection, and score tracking.

### Game Grid:

The game environment is displayed as a 10 × 10 grid, where:

- P represents the player
- E represents enemies
- . represents empty spaces

## SNAPSHOTS:



```
final case study > C: project.cpp > Game > checkCollision()
1  #include <iostream>
2  using namespace std;
3
4  // Width and height of the game grid
5  const int WIDTH = 10;
6  const int HEIGHT = 10;
7
8  // Base class representing any object in the game (player or enemy)
9  class GameObject
10 {
11 public:
12     int x, y; // position of the object on the grid
13     char symbol; // character used to display the object
14
15     // Constructor to initialize object position and symbol
16     GameObject(int x, int y, char symbol)
17     {
18         this->x = x;
19         this->y = y;
20         this->symbol = symbol;
21     }
22 };
23
24 // Main Game class which controls the game logic
25 class Game
26 {
27 private:
28     bool isRunning; // controls whether the game loop runs or stops
29     int score; // player's score
30     GameObject player; // player object
31     vector<GameObject> enemies; // list storing enemy objects
32
33 public:
34     // Constructor initializes player and enemies
35     Game() : player(WIDTH / 2, HEIGHT / 2, 'P') // player starts in center
36     {
37         isRunning = true; // game starts in running state
38         score = 0; // initial score
39
40         // Creating 3 enemies at random positions on the grid
41         for (int i = 0; i < 3; i++)
42         {
43             enemies.push_back(GameObject(rand() % WIDTH, rand() % HEIGHT, 'E'));
44         }
45     }
46 }
```

```
operoverloading.cpp  project.cpp mini project  project.cpp final case study X  file.cpp  2.cpp
final case study > project.cpp > Game > checkCollision()
25  class Game
47      // Main game loop
48  void run()
49  {
50      while (isRunning) // loop runs until user quits
51      {
52          render(); // display the grid
53          processInput(); // take user input
54          update(); // update game state
55      }
56  }
57
58  // Function to take player movement input
59  void processInput()
60  {
61      char input;
62
63      cout << "\nMove (W A S D) | Q Quit : ";
64      cin >> input;
65
66      // Quit game
67      if (input == 'q' || input == 'Q')
68          isRunning = false;
69
70      // Move player up
71      else if (input == 'w' || input == 'W')
72          player.y--;
73
74      // Move player down
75      else if (input == 's' || input == 'S')
76          player.y++;
77
78      // Move player left
79      else if (input == 'a' || input == 'A')
80          player.x--;
81
82      // Move player right
83      else if (input == 'd' || input == 'D')
84          player.x++;
85  }
86
87  // Updates the game state
88  void update()
89  {
90      // Prevent player from going outside the grid boundaries
```

```
operoverloading.cpp  project.cpp mini project  project.cpp final case study X  file.cpp  2.cpp
final case study > project.cpp > Game > checkCollision()
25  class Game
149  void render()
150  {
151      for (int y = 0; y < HEIGHT; y++)
152      {
153          for (int x = 0; x < WIDTH; x++)
154          {
155              // Print player if player is at this position
156              if (player.x == x && player.y == y)
157              {
158                  cout << "P ";
159                  printed = true;
160              }
161
162              // Check if any enemy is at this position
163              for (auto &enemy : enemies)
164              {
165                  if (enemy.x == x && enemy.y == y)
166                  {
167                      cout << "E ";
168                      printed = true;
169                      break;
170                  }
171              }
172
173              // If nothing is at this position, print empty cell
174              if (!printed)
175                  cout << ". ";
176          }
177
178          cout << endl; // move to next row
179      }
180  }
181
182  // Program entry point
183  int main()
184  {
185      srand(time(0)); // seed random number generator
186
187      Game game; // create game object
188      game.run(); // start the game loop
189
190      cout << "\nGame Over! Final Score: " << endl;
191
192      return 0; // end program
193  }
```

**Title: C++ Program Implementation:**

